

Inigo Quilez :: articles :: intersectors



Intro

This is a collection of ray-surface intersectors for some common primitive types. You can use them to implement a pathtracer, UI clicking, sound propagation, collision detection or anything that requires analytic raycasting. I have derived these myself, and while I have used them professionally, I haven't put them to the most strict of the tests. If you notice precision errors in any of them you can choose to improve these implementations (at some performance cost) or improve how your renderer generally handles precision problems. When possible, I've tried to get the number of operations to a minimum (for example the capped cone and capsule intersectors only check against one of the caps, even though these primitives have two).

Note that all the primitives listed here come with a link to a realtime online demo in Shadertoy. In fact, this public playlist contains all the shadertoy examples: <https://www.shadertoy.com/playlist/l3dXRf>, so don't miss that out.

Primitives

In all the examples below, **ro** is the ray origin, and **rd** is the ray direction. The functions always return the distance to the closest intersection, and sometimes some additional information (second intersection, normal or barycentric coordinates). Most primitives are centered at the origin (you'll need to transform the ray origin and direction accordingly), but some of the functions above accept an arbitrary primitive location and orientation when such implementation is faster than performing a ray transformation. Most of these functions only work for a ray with origin outside of the primitive. Fixing that shouldn't be difficult.



Sphere

[Shadertoy example](#)

```
// sphere of size ra centered at point ce
vec2 sphIntersect( in vec3 ro, in vec3 rd, in vec3 ce, float ra )
{
    vec3 oc = ro - ce;
    float b = dot( oc, rd );
    float c = dot( oc, oc ) - ra*ra;
    float h = b*b - c;
    if( h<0.0 ) return vec2(-1.0); // no intersection
```

```

    h = sqrt( h );
    return vec2( -b-h, -b+h );
}

```

Alternative method computes **h** (the squared distance from the closest ray point to the sphere, **qc** below) with a projection rather than by using Pythagoras' theorem. This is less precision hungry because we don't generate large numbers (in comparison to the size of the sphere) since we don't square triangle edges (**b*b**):

```

// sphere of size ra centered at point ce
vec2 sphIntersect( in vec3 ro, in vec3 rd, in vec3 ce, float ra )
{
    vec3 oc = ro - ce;
    float b = dot( oc, rd );
    vec3 qc = oc - b*rd;
    float h = ra*ra - dot( qc, qc );
    if( h<0.0 ) return vec2(-1.0); // no intersection
    h = sqrt( h );
    return vec2( -b-h, -b+h );
}

```

The functions return the entry and exit point of the ray with the sphere in X and Y, and marks the exit (Y) with a negative value if there isn't any intersection. So,

```

vec2 t = sphIntersect( ro, rd, center, radius );

    if( t.y<0.0 ) { } // ray does NOT intersect the sphere
else if( t.x<0.0 ) { } // ro inside the sphere, t.y is intersection distance
else          { } // ro outside the sphere, t.x is intersection distance

```



Box

Shadertoy example

```

// axis aligned, center at the origin, dimensions "boxSize"
vec2 boxIntersection( in vec3 ro, in vec3 rd, vec3 boxSize, out vec3 oNormal )
{
    vec3 m = 1.0/rd; // can precompute if traversing a set of aligned boxes
    vec3 n = m*ro;   // can precompute if traversing a set of aligned boxes
    vec3 k = abs(m)*boxSize;
    vec3 t1 = -n - k;
    vec3 t2 = -n + k;
    float tN = max( max( t1.x, t1.y ), t1.z );
    float tF = min( min( t2.x, t2.y ), t2.z );
    if( tN>tF || tF<0.0 ) return vec2(-1.0); // no intersection
    oNormal = (tN>0.0) ? step(vec3(tN),t1)) : // ro outside the box
                step(t2,vec3(tF)); // ro inside the box
    oNormal *= -sign(rd);
}

```

```

    return vec2( tN, tF );
}

```



Rounded Box

Shadertoy example

```

// axis aligned, center at the origin, dimensions "size", extrusion by "rad"
float roundedboxIntersect( in vec3 ro, in vec3 rd, in vec3 size, in float rad )
{
    // bounding box
    vec3 m = 1.0/rd;
    vec3 n = m*ro;
    vec3 k = abs(m)*(size+rad);
    vec3 t1 = -n - k;
    vec3 t2 = -n + k;
    float tN = max( max( t1.x, t1.y ), t1.z );
    float tF = min( min( t2.x, t2.y ), t2.z );
    if( tN>tF || tF<0.0 ) return -1.0;
    float t = tN;

    // convert to first octant
    vec3 pos = ro+t*rd;
    vec3 s = sign(pos);
    ro *= s;
    rd *= s;
    pos *= s;

    // faces
    pos -= size;
    pos = max( pos.xyz, pos.yzx );
    if( min(min(pos.x,pos.y),pos.z) < 0.0 ) return t;

    // some precomputation
    vec3 oc = ro - size;
    vec3 dd = rd*rd;
    vec3 oo = oc*oc;
    vec3 od = oc*rd;
    float ra2 = rad*rad;

    t = 1e20;

    // corner
    {
        float b = od.x + od.y + od.z;
        float c = oo.x + oo.y + oo.z - ra2;
        float h = b*b - c;
        if( h>0.0 ) t = -b-sqrt(h);
    }

    // edge x
    {
        float a = dd.y + dd.z;

```

```

float b = od.y + od.z;
float c = oo.y + oo.z - ra2;
float h = b*b - a*c;
if( h>0.0 )
{
    h = (-b-sqrt(h))/a;
    if( h>0.0 && h<t && abs(ro.x+rd.x*h)<size.x ) t = h;
}
}

// edge y
{
float a = dd.z + dd.x;
float b = od.z + od.x;
float c = oo.z + oo.x - ra2;
float h = b*b - a*c;
if( h>0.0 )
{
    h = (-b-sqrt(h))/a;
    if( h>0.0 && h<t && abs(ro.y+rd.y*h)<size.y ) t = h;
}
}

// edge z
{
float a = dd.x + dd.y;
float b = od.x + od.y;
float c = oo.x + oo.y - ra2;
float h = b*b - a*c;
if( h>0.0 )
{
    h = (-b-sqrt(h))/a;
    if( h>0.0 && h<t && abs(ro.z+rd.z*h)<size.z ) t = h;
}
}

if( t>1e19 ) t=-1.0;

return t;
}

// normal of a rounded box
vec3 roundedboxNormal( in vec3 pos, in vec3 siz, in float rad )
{
    return sign(pos)*normalize(max(abs(pos)-siz,0.0));
}

// plane defined by p (p.xyz must be normalized)
float plaIntersect( in vec3 ro, in vec3 rd, in vec4 p )
{
    return -(dot(ro,p.xyz)+p.w)/dot(rd,p.xyz);
}

```



Plane



Disk

Shadertoy example

```
// disk center c, normal n, radius r
float diskIntersect( in vec3 ro, in vec3 rd, vec3 c, vec3 n, float r )
{
    vec3 o = ro - c;
    float t = -dot(n,o)/dot(rd,n);
    vec3 q = o + rd*t;
    return (dot(q,q)<r*r) ? t : -1.0;
}
```



Hexagonal Prism

Shadertoy example

```
// returns t and normal
vec4 iHexPrism( in vec3 ro, in vec3 rd, in float ra, in float he )
{
    const float ks3 = 0.866025;

    // normals
    const vec3 n1 = vec3( 1.0,0.0,0.0);
    const vec3 n2 = vec3( 0.5,0.0,ks3);
    const vec3 n3 = vec3(-0.5,0.0,ks3);
    const vec3 n4 = vec3( 0.0,1.0,0.0);

    // slabs intersections
    vec3 t1 = vec3((vec2(ra,-ra)-dot(ro,n1))/dot(rd,n1), 1.0);
    vec3 t2 = vec3((vec2(ra,-ra)-dot(ro,n2))/dot(rd,n2), 1.0);
    vec3 t3 = vec3((vec2(ra,-ra)-dot(ro,n3))/dot(rd,n3), 1.0);
    vec3 t4 = vec3((vec2(he,-he)-dot(ro,n4))/dot(rd,n4), 1.0);

    // inetsection selection
    if( t1.y<t1.x ) t1=vec3(t1.yx,-1.0);
    if( t2.y<t2.x ) t2=vec3(t2.yx,-1.0);
    if( t3.y<t3.x ) t3=vec3(t3.yx,-1.0);
    if( t4.y<t4.x ) t4=vec3(t4.yx,-1.0);

    vec4 tN=vec4(t1.x,t1.z*n1);
    if( t2.x>tN.x ) tN=vec4(t2.x,t2.z*n2);
    if( t3.x>tN.x ) tN=vec4(t3.x,t3.z*n3);
    if( t4.x>tN.x ) tN=vec4(t4.x,t4.z*n4);

    float tF = min(min(t1.y,t2.y),min(t3.y,t4.y));
```

```

// no intersection
if( tN.x>tF || tF<0.0) return vec4(-1.0);

return tN; // return tF too for exit point
}

```



Wedge

Shadertoy example

```

// returns t and normal
vec4 iWedge( in vec3 ro, in vec3 rd, in vec3 s )
{
    // intersect plane box
    vec3 m = 1.0/rd;
    vec3 z = vec3(rd.x>=0.0?1.0:-1.0, rd.y>=0.0?1.0:-1.0, rd.z>=0.0?1.0:-1.0);
    vec3 k = s*z;
    vec3 t1 = (-ro - k)*m;
    vec3 t2 = (-ro + k)*m;
    float tn = max(max(t1.x, t1.y), t1.z);
    float tf = min(min(t2.x, t2.y), t2.z);
    if( tn>tf ) return vec4(-1.0);

    // boolean with plane
    float k1 = s.y*ro.x - s.x*ro.y;
    float k2 = s.x*rd.y - s.y*rd.x;
    float tp = k1/k2;

    if( k1>tn*k2 )
        return vec4(tn,-step(tn,t1)*z); // box
    if( tp>tn && tp<tf )
        return vec4(tp,normalize(vec3(-s.y,s.x,0.0))); // plane
    return vec4(-1.0);
}

```



Capsule

Shadertoy example

```

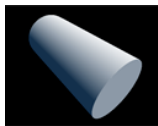
// capsule defined by extremes pa and pb, and radius ra
// Note that only ONE of the two spherical caps is checked for
// intersections, which is a nice optimization
float capIntersect( vec3 ro, vec3 rd, vec3 pa, vec3 pb, float ra )
{
    vec3 ba = pb - pa;
    vec3 oa = ro - pa;
    float baba = dot(ba,ba);

```

```

float bard = dot(ba,rd);
float baoa = dot(ba,oa);
float rdoa = dot(rd,oa);
float oaoa = dot(oa,oa);
float a = baba - bard*bard;
float b = baba*rdoa - baoa*bard;
float c = baba*oaoa - baoa*baoa - ra*ra*baba;
float h = b*b - a*c;
if( h >= 0.0 )
{
    float t = (-b-sqrt(h))/a;
    float y = baoa + t*bard;
    // body
    if( y>0.0 && y<baba ) return t;
    // caps
    vec3 oc = (y <= 0.0) ? oa : ro - pb;
    b = dot(rd,oc);
    c = dot(oc,oc) - ra*ra;
    h = b*b - c;
    if( h>0.0 ) return -b - sqrt(h);
}
return -1.0;
}

```



Capped cylinder

Shadertoy example

```

// cylinder defined by extremes a and b, and radius ra
vec4 cylIntersect( in vec3 ro, in vec3 rd, in vec3 a, in vec3 b, float ra )
{
    vec3 ba = b - a;
    vec3 oc = ro - a;
    float baba = dot(ba,ba);
    float bard = dot(ba,rd);
    float baoc = dot(ba,oc);
    float k2 = baba - bard*bard;
    float k1 = baba*dot(oc,rd) - baoc*bard;
    float k0 = baba*dot(oc,oc) - baoc*baoc - ra*ra*baba;
    float h = k1*k1 - k2*k0;
    if( h<0.0 ) return vec4(-1.0); //no intersection
    h = sqrt(h);
    float t = (-k1-h)/k2;
    // body
    float y = baoc + t*bard;
    if( y>0.0 && y<baba ) return vec4( t, (oc+t*rd - ba*y/baba)/ra );
    // caps
    t = ( (y<0.0) ? 0.0 : baba ) - baoc)/bard;
    if( abs(k1+k2*t)<h )
    {
        return vec4( t, ba*sign(y)/sqrt(baba) );
    }
}

```

```

    return vec4(-1.0); //no intersection
}

// normal at point p of cylinder (a,b,ra), see above
vec3 cylNormal( in vec3 p, in vec3 a, in vec3 b, float ra )
{
    vec3 pa = p - a;
    vec3 ba = b - a;
    float baba = dot(ba,ba);
    float paba = dot(pa,ba);
    float h = dot(pa,ba)/baba;
    return (pa - ba*h)/ra;
}

```



Cylinder

```

// infinite cylinder with base point "cb", normalized axis "ca" and radius "cr"
vec2 cylIntersect( in vec3 ro, in vec3 rd, in vec3 cb, in vec3 ca, float cr )
{
    vec3 oc = ro - cb;
    float card = dot(ca,rd);
    float caoc = dot(ca,oc);
    float a = 1.0 - card*card;
    float b = dot( oc, rd ) - caoc*card;
    float c = dot( oc, oc ) - caoc*caoc - cr*cr;
    float h = b*b - a*c;
    if( h<0.0 ) return vec2(-1.0); //no intersection
    h = sqrt(h);
    return vec2(-b-h,-b+h)/a;
}

```



Capped cone

Shadertoy example

```

// cone defined by extremes "pa" and "pb", and radius "ra" and "rb"
// Only one square root and one division in the worst case. dot2(v) is dot(v,v)
vec4 coneIntersect( vec3 ro, vec3 rd, vec3 pa, vec3 pb, float ra, float rb )
{
    vec3 ba = pb - pa;
    vec3 oa = ro - pa;
    vec3 ob = ro - pb;
    float m0 = dot(ba,ba);
    float m1 = dot(oa,ba);

```

```

float m2 = dot(rd,ba);
float m3 = dot(rd,oa);
float m5 = dot(oa,oa);
float m9 = dot(ob,ba);

// caps
if( m1<0.0 )
{
    if( dot2(oa*m2-rd*m1)<(ra*ra*m2*m2) ) // delayed division
        return vec4(-m1/m2,-ba*inversesqrt(m0));
}
else if( m9>0.0 )
{
    float t = -m9/m2; // NOTE delayed division
    if( dot2(ob+rd*t)<(rb*rb) )
        return vec4(t,ba*inversesqrt(m0));
}

// body
float rr = ra - rb;
float hy = m0 + rr*rr;
float k2 = m0*m0 - m2*m2*hy;
float k1 = m0*m0*m3 - m1*m2*hy + m0*ra*(rr*m2*1.0 );
float k0 = m0*m0*m5 - m1*m1*hy + m0*ra*(rr*m1*2.0 - m0*ra);
float h = k1*k1 - k2*k0;
if( h<0.0 ) return vec4(-1.0); //no intersection
float t = (-k1-sqrt(h))/k2;
float y = m1 + t*m2;
if( y<0.0 || y>m0 ) return vec4(-1.0); //no intersection
return vec4(t, normalize(m0*(m0*(oa+t*rd)+rr*ba*ra)-ba*hy*y));
}

```



Rounded cone

Shadertoy example

```

// cone defined by extremes "pa" and "pb", and radius "ra" and "rb"
vec4 iRoundedCone( vec3 ro, vec3 rd, vec3 pa, vec3 pb, float ra, float rb )
{
    vec3 ba = pb - pa;
    vec3 oa = ro - pa;
    vec3 ob = ro - pb;
    float rr = ra - rb;
    float m0 = dot(ba,ba);
    float m1 = dot(ba,oa);
    float m2 = dot(ba,rd);
    float m3 = dot(rd,oa);
    float m5 = dot(oa,oa);
    float m6 = dot(ob,rd);
    float m7 = dot(ob,ob);

    // body

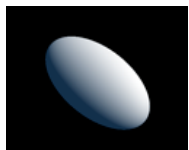
```

```

float d2 = m0-rr*rr;
float k2 = d2 - m2*m2;
float k1 = d2*m3 - m1*m2 + m2*rr*ra;
float k0 = d2*m5 - m1*m1 + m1*rr*ra*2.0 - m0*ra*ra;
float h = k1*k1 - k0*k2;
if( h<0.0) return vec4(-1.0);
float t = (-sqrt(h)-k1)/k2;
//if( t<0.0 ) return vec4(-1.0);
float y = m1 - ra*rr + t*m2;
if( y>0.0 && y<d2 ) return vec4(t, normalize(d2*(oa+t*rd)-ba*y));

// caps
float h1 = m3*m3 - m5 + ra*ra;
float h2 = m6*m6 - m7 + rb*rb;
if( max(h1,h2)<0.0 ) return vec4(-1.0);
vec4 r = vec4(1e20);
if( h1>0.0 )
{
    t = -m3 - sqrt( h1 );
    r = vec4( t, (oa+t*rd)/ra );
}
if( h2>0.0 )
{
    t = -m6 - sqrt( h2 );
    if( t<r.x )
        r = vec4( t, (ob+t*rd)/rb );
}
return r;
}

```



Ellipsoid

Shadertoy example

```

// ellipsoid centered at the origin with radii ra
vec2 eliIntersect( in vec3 ro, in vec3 rd, in vec3 ra )
{
    vec3 ocn = ro/ra;
    vec3 rdn = rd/ra;
    float a = dot( rdn, rdn );
    float b = dot( ocn, rdn );
    float c = dot( ocn, ocn );
    float h = b*b - a*(c-1.0);
    if( h<0.0 ) return vec2(-1.0); //no intersection
    h = sqrt(h);
    return vec2(-b-h,-b+h)/a;
}

vec3 eliNormal( in vec3 pos, in vec3 ra )
{

```

```

    return normalize( pos/(ra*ra) );
}

```



Triangle

Shadertoy example

```

// triangle defined by vertices v0, v1 and v2
vec3 triIntersect( in vec3 ro, in vec3 rd, in vec3 v0, in vec3 v1, in vec3 v2 )
{
    vec3 v1v0 = v1 - v0;
    vec3 v2v0 = v2 - v0;
    vec3 rov0 = ro - v0;
    vec3 n = cross( v1v0, v2v0 );
    vec3 q = cross( rov0, rd );
    float d = 1.0/dot( rd, n );
    float u = d*dot( -q, v2v0 );
    float v = d*dot( q, v1v0 );
    float t = d*dot( -n, rov0 );
    if( u<0.0 || v<0.0 || (u+v)>1.0 ) t = -1.0;
    return vec3( t, u, v );
}

```



Ellipse

Shadertoy example

```

// ellipse defined by its center c and its two radii u and v
vec3 ellIntersect( in vec3 ro, in vec3 rd, vec3 c, vec3 u, vec3 v )
{
    vec3 q = ro - c;
    vec3 n = cross(u,v);
    float t = -dot(n,q)/dot(rd,n);
    float r = dot(u,q + rd*t);
    float s = dot(v,q + rd*t);
    if( r*r+s*s>1.0 ) return vec3(-1.0); //no intersection
    return vec3(t,s,r);
}

vec3 ellNormal( in vec2 u, vec2 v )
{
    return normalize( cross(u,v) );
}

```

Torus

Shadertoy example

```

float torIntersect( in vec3 ro, in vec3 rd, in vec2 tor )
{
    float po = 1.0;
    float Ra2 = tor.x*tor.x;
    float ra2 = tor.y*tor.y;
    float m = dot(ro,ro);
    float n = dot(ro,rd);
    float k = (m + Ra2 - ra2)/2.0;
    float k3 = n;
    float k2 = n*n - Ra2*dot(rd.xy,rd.xy) + k;
    float k1 = n*k - Ra2*dot(rd.xy,ro.xy);
    float k0 = k*k - Ra2*dot(ro.xy,ro.xy);

    if( abs(k3*(k3*k3-k2)+k1) < 0.01 )
    {
        po = -1.0;
        float tmp=k1; k1=k3; k3=tmp;
        k0 = 1.0/k0;
        k1 = k1*k0;
        k2 = k2*k0;
        k3 = k3*k0;
    }

    float c2 = k2*2.0 - 3.0*k3*k3;
    float c1 = k3*(k3*k3-k2)+k1;
    float c0 = k3*(k3*(c2+2.0*k2)-8.0*k1)+4.0*k0;
    c2 /= 3.0;
    c1 *= 2.0;
    c0 /= 3.0;
    float Q = c2*c2 + c0;
    float R = c2*c2*c2 - 3.0*c2*c0 + c1*c1;
    float h = R*R - Q*Q*Q;

    if( h>=0.0 )
    {
        h = sqrt(h);
        float v = sign(R+h)*pow(abs(R+h),1.0/3.0); // cube root
        float u = sign(R-h)*pow(abs(R-h),1.0/3.0); // cube root
        vec2 s = vec2( (v+u)+4.0*c2, (v-u)*sqrt(3.0));
        float y = sqrt(0.5*(length(s)+s.x));
        float x = 0.5*s.y/y;
        float r = 2.0*c1/(x*x+y*y);
        float t1 = x - r - k3; t1 = (po<0.0)?2.0/t1:t1;
        float t2 = -x - r - k3; t2 = (po<0.0)?2.0/t2:t2;
        float t = 1e20;
        if( t1>0.0 ) t=t1;
        if( t2>0.0 ) t=min(t,t2);
        return t;
    }

    float sQ = sqrt(Q);
    float w = sQ*cos( acos(-R/(sQ*Q)) / 3.0 );

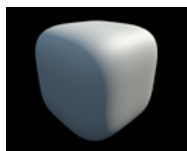
```

```

float d2 = -(w+c2); if( d2<0.0 ) return -1.0;
float d1 = sqrt(d2);
float h1 = sqrt(w - 2.0*c2 + c1/d1);
float h2 = sqrt(w - 2.0*c2 - c1/d1);
float t1 = -d1 - h1 - k3; t1 = (po<0.0)?2.0/t1:t1;
float t2 = -d1 + h1 - k3; t2 = (po<0.0)?2.0/t2:t2;
float t3 = d1 - h2 - k3; t3 = (po<0.0)?2.0/t3:t3;
float t4 = d1 + h2 - k3; t4 = (po<0.0)?2.0/t4:t4;
float t = 1e20;
if( t1>0.0 ) t=t1;
if( t2>0.0 ) t=min(t,t2);
if( t3>0.0 ) t=min(t,t3);
if( t4>0.0 ) t=min(t,t4);
return t;
}

vec3 torNormal( in vec3 pos, vec2 tor )
{
    return normalize( pos*(dot(pos,pos)-tor.y*tor.y - tor.x*tor.x*vec3(1.0,1.0,-1.0)));
}

```



Sphere 4

Shadertoy example

```

float sph4Intersect( in vec3 ro, in vec3 rd, in float ra )
{
    float r2 = ra*ra;
    vec3 d2 = rd*rd; vec3 d3 = d2*rd;
    vec3 o2 = ro*ro; vec3 o3 = o2*ro;
    float ka = 1.0/dot(d2,d2);
    float k3 = ka* dot(ro,d3);
    float k2 = ka* dot(o2,d2);
    float k1 = ka* dot(o3,rd);
    float k0 = ka*(dot(o2,o2) - r2*r2);
    float c2 = k2 - k3*k3;
    float c1 = k1 + 2.0*k3*k3*k3 - 3.0*k3*k2;
    float c0 = k0 - 3.0*k3*k3*k3*k3 + 6.0*k3*k3*k2 - 4.0*k3*k1;
    float p = c2*c2 + c0/3.0;
    float q = c2*c2*c2 - c2*c0 + c1*c1;
    float h = q*q - p*p*p;
    if( h<0.0 ) return -1.0; //no intersection
    float sh = sqrt(h);
    float s = sign(q+sh)*pow(abs(q+sh),1.0/3.0); // cuberoot
    float t = sign(q-sh)*pow(abs(q-sh),1.0/3.0); // cuberoot
    vec2 w = vec2( s+t,s-t );
    vec2 v = vec2( w.x+c2*4.0, w.y*sqrt(3.0) )*0.5;
    float r = length(v);
    return -abs(v.y)/sqrt(r+v.x) - c1/r - k3;
}

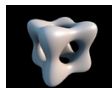
vec3 sph4Normal( in vec3 pos )

```

```

{
    return normalize( pos*pos*pos );
}

```



Goursat

Shadertoy example

```

float gouIntersect( in vec3 ro, in vec3 rd, in float ka, float kb )
{
    float po = 1.0;
    vec3 rd2 = rd*rd; vec3 rd3 = rd2*rd;
    vec3 ro2 = ro*ro; vec3 ro3 = ro2*ro;
    float k4 = dot(rd2,rd2);
    float k3 = dot(ro ,rd3);
    float k2 = dot(ro2,rd2) - kb/6.0;
    float k1 = dot(ro3,rd ) - kb*dot(rd,ro)/2.0;
    float k0 = dot(ro2,ro2) + ka - kb*dot(ro,ro);
    k3 /= k4;
    k2 /= k4;
    k1 /= k4;
    k0 /= k4;
    float c2 = k2 - k3*(k3);
    float c1 = k1 + k3*(2.0*k3*k3-3.0*k2);
    float c0 = k0 + k3*(k3*(c2+k2)*3.0-4.0*k1);

    if( abs(c1) < 0.1*abs(c2) )
    {
        po = -1.0;
        float tmp=k1; k1=k3; k3=tmp;
        k0 = 1.0/k0;
        k1 = k1*k0;
        k2 = k2*k0;
        k3 = k3*k0;
        c2 = k2 - k3*(k3);
        c1 = k1 + k3*(2.0*k3*k3-3.0*k2);
        c0 = k0 + k3*(k3*(c2+k2)*3.0-4.0*k1);
    }

    c0 /= 3.0;
    float Q = c2*c2 + c0;
    float R = c2*c2*c2 - 3.0*c0*c2 + c1*c1;
    float h = R*R - Q*Q*Q;

    if( h>0.0 ) // 2 intersections
    {
        h = sqrt(h);
        float s = sign(R+h)*pow(abs(R+h),1.0/3.0); // cube root
        float u = sign(R-h)*pow(abs(R-h),1.0/3.0); // cube root
        float x = s+u+4.0*c2;
        float y = s-u;
        float ks = x*x + y*y*3.0;
    }
}

```

```

    float k = sqrt(ks);
    float t = -0.5*po*abs(y)*sqrt(6.0/(k+x)) - 2.0*c1*(k+x)/(ks+x*k) - k3;
    return (po<0.0)?1.0/t:t;
}

// 4 intersections
float sQ = sqrt(Q);
float w = sQ*cos(acos(-R/(sQ*Q))/3.0);
float d2 = -w - c2;
if( d2<0.0 ) return -1.0; //no intersection
float d1 = sqrt(d2);
float h1 = sqrt(w - 2.0*c2 + c1/d1);
float h2 = sqrt(w - 2.0*c2 - c1/d1);
float t1 = -d1 - h1 - k3; t1 = (po<0.0)?1.0/t1:t1;
float t2 = -d1 + h1 - k3; t2 = (po<0.0)?1.0/t2:t2;
float t3 = d1 - h2 - k3; t3 = (po<0.0)?1.0/t3:t3;
float t4 = d1 + h2 - k3; t4 = (po<0.0)?1.0/t4:t4;
float t = 1e20;
if( t1>0.0 ) t=t1;
if( t2>0.0 ) t=min(t,t2);
if( t3>0.0 ) t=min(t,t3);
if( t4>0.0 ) t=min(t,t4);
return t;
}

vec3 gouNormal( in vec3 pos, float ka, float kb )
{
    return normalize( 4.0*pos*pos*pos - 2.0*pos*kb );
}

```

inigo quilez - learning computer graphics since 1994